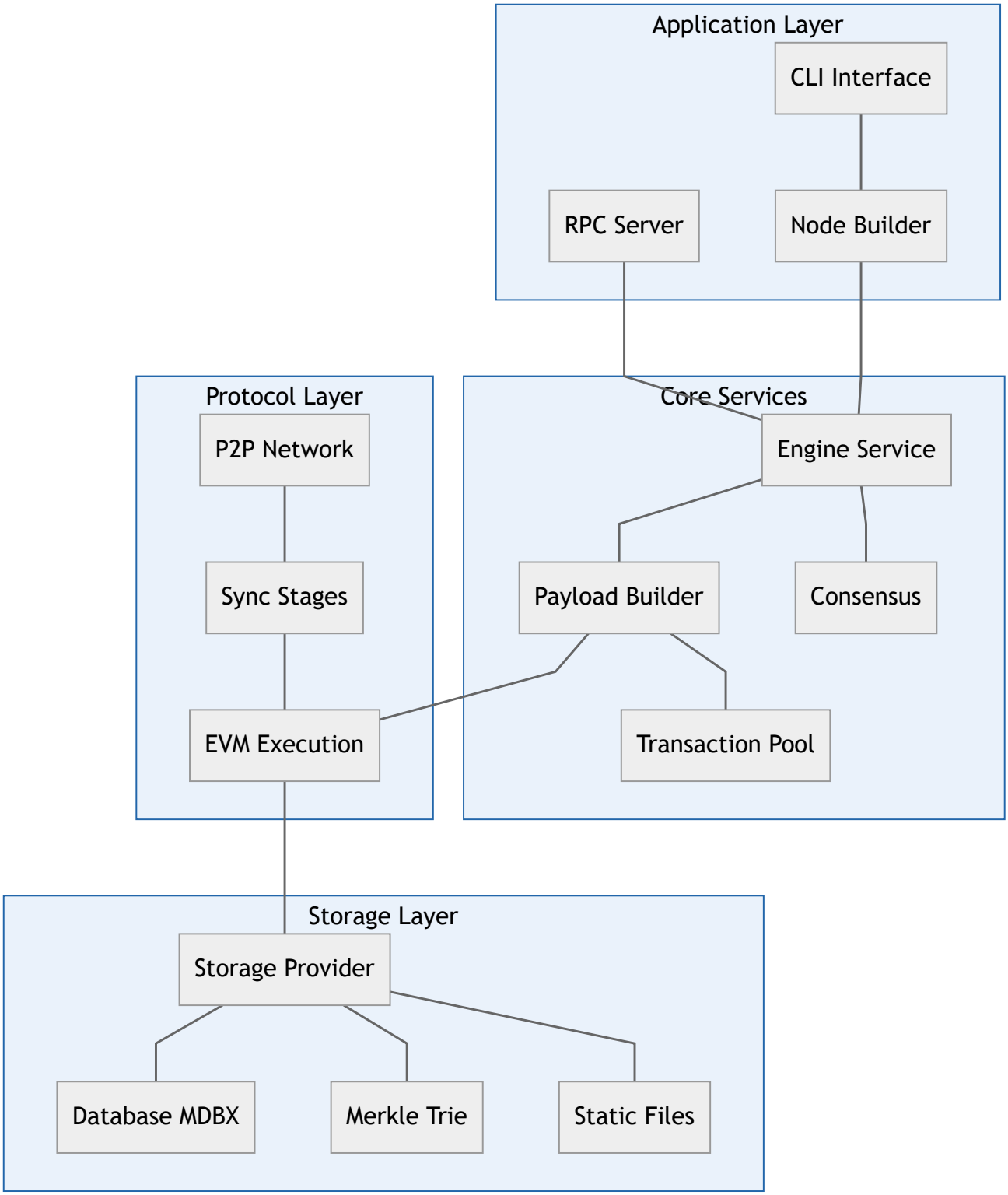
Reth 전체 아키텍처 개요

소개

Reth는 Rust로 구현된 모듈화된 Ethereum 클라이언트로, 150개 이상의 크레이트로 구성되어 있습니다. 각 컴포넌트는 명확한 책임을 가지며, 트레이트 기반의 추상화를 통해 유연하고 확장 가능한 아키텍처를 제공합니다.

전체 시스템 아키텍처



핵심 디렉터리 구조 및 역할

1. 노드 관리 시스템 (`crates/node/`)

| 디렉터리 | 역할 | 핵심 구조체 | 주요 함수 |

|----------|------|----------|----------|

| `node/api/` | 노드 API 추상화 | `NodeTypes`, `FullNodeComponents` | `components()`, `provider()` |

| `node/builder/` | 노드 구성 빌더 | `NodeBuilder`, `ComponentsBuilder` | `with_types()`, `build()` |

| `node/core/` | 노드 핵심 유틸리티 | `NodeConfig`, `ChainSpec` | `load_config()`, `init()` |

| `node/types/` | 노드 타입 정의 | `NodeTypesWithEngine` | `primitives()` |

주요 책임: Ethereum 노드의 전체 라이프사이클 관리, 컴포넌트 조립 및 설정

2. 스토리지 시스템 (`crates/storage/`)

| 디렉터리 | 역할 | 핵심 구조체 | 주요 함수 |

|----------|------|----------|----------|

| `storage/provider/` | 데이터 접근 추상화 | `ProviderFactory`, `DatabaseProvider` | `provider()`, `latest()` |

| `storage/db/` | 데이터베이스 구현 | `DatabaseEnv`, `Tables` | `open()`, `create_cursor()` |

| `storage/db-api/` | DB 추상화 인터페이스 | `Database`, `DbTx` | `tx()`, `view()` |

| `storage/libmdbx-rs/` | MDBX 바인딩 | `Environment`, `Transaction` | `begin_rw_txn()`, `commit()` |

| `storage/codecs/` | 직렬화/역직렬화 | `Compact`, `Encode`, `Decode` | `encode()`, `decode()` |

| `static-file/` | 정적 파일 관리 | `StaticFileProvider`, `NippyJar` | `get_segment()`, `append()` |

주요 책임: 블록체인 데이터의 영구 저장, 효율적인 데이터 접근, 압축 및 아카이빙

3. 네트워킹 시스템 (`crates/net/`)

| 디렉터리 | 역할 | 핵심 구조체 | 주요 함수 |

|----------|------|----------|----------|

| net/network/ | P2P 네트워크 관리 | NetworkManager, NetworkHandle | spawn(), peers_info() |

| net/p2p/ | P2P 프로토콜 구현 | P2PStream, ProtocolHandler | handle_message(), send() |

| net/eth-wire/ | Ethereum 프로토콜 | EthStream, EthMessage | send_message(), recv() |

| net/discv4/ | 피어 발견 v4 | Discv4, NodeRecord | lookup(), ping() |

| net/discv5/ | 피어 발견 v5 | Discv5, Enr | find_node(), talk_req() |

| net/downloaders/ | 데이터 다운로드 | HeaderDownloader, BodyDownloader | download(), stream_headers() |

주요 책임: 피어 발견, 연결 관리, 메시지 전송/수신, 데이터 동기화

4. 실행 엔진 (`crates/evm/`, `crates/engine/`)

| 디렉터리 | 역할 | 핵심 구조체 | 주요 함수 |

|---|---|---|---|

| evm/evm/ | EVM 추상화 | ConfigureEvm, EvmFactory | evm_with_env(), executor() |

| evm/execution-types/ | 실행 타입 | ExecutionOutcome, BundleState | state_root(), apply_to_state() |

| engine/service/ | Engine API 서비스 | EngineService, EngineHandler | new_payload(), fork_choice_updated() |

| engine/tree/ | 블록 트리 관리 | EngineTree, BlockBuffer | insert_block(), make_canonical() |

| revm/ | revm 통합 | RethEvmConfig, RethExecutor | configure_evm(), execute_block() |

주요 책임: 트랜잭션 실행, 상태 변경, 블록 검증, Engine API 구현

5. 동기화 시스템 (`crates/stages/`)

| 디렉터리 | 역할 | 핵심 구조체 | 주요 함수 |

|---|---|---|---|

| `stages/stages/` | 동기화 단계 구현 | `Pipeline`, `Stage` | `execute()`, `unwind()` |

| `stages/api/` | 동기화 API | `StageCheckpoint`, `ExecInput` | `stage_checkpoint()`, `commit()` |

| `stages/types/` | 동기화 타입 | `StageId`, `BlockNumber` | `stage_range()` |

주요 책임: 단계별 블록체인 동기화, 체크포인트 관리, 되감기 지원

6. RPC 서비스 (`crates/rpc/`)

| 디렉터리 | 역할 | 핵심 구조체 | 주요 함수 |

|---|---|---|---|

| `rpc/rpc-api/` | RPC API 정의 | `EthApiServer`, `EngineApiServer` | `block_number()`, `send_raw_transaction()` |

| `rpc/rpc-eth-api/` | Ethereum RPC 구현 | `EthApi`, `EthFilter` | `get_block()`, `get_transaction()` |

| `rpc/rpc-engine-api/` | Engine API 구현 | `EngineApi` | `new_payload_v3()`, `get_payload_v3()` |

| `rpc/rpc-builder/` | RPC 서버 빌더 | `RpcServerBuilder` | `build()`, `start()` |

주요 책임: JSON-RPC 서버, Ethereum API, Engine API, WebSocket 지원

7. 트랜잭션 풀 (`crates/transaction-pool/`)

| 디렉터리 | 역할 | 핵심 구조체 | 주요 함수 |

|---|---|---|---|

| `transaction-pool/` | 트랜잭션 풀 관리 | `Pool`, `TxPool` | `add_transaction()`, `best_transactions()` |

| `transaction-pool/validate/` | 트랜잭션 검증 | `EthTransactionValidator` | `validate_transaction()` |

| `transaction-pool/ordering/` | 트랜잭션 정렬 | `CoinbaseTipOrdering` | `priority()`, `compare()` |

주요 책임: 트랜잭션 검증, 메모리 풀 관리, 우선순위 정렬, 가스 가격 정책

8. 합의 메커니즘 (`crates/consensus/`)

| 디렉터리 | 역할 | 핵심 구조체 | 주요 함수 |

|---|---|---|---|

| `consensus/consensus/` | 합의 추상화 | `Consensus`, `HeaderValidator` | `validate_header()`, `validate_block()` |

| `ethereum/consensus/` | Ethereum 합의 | `EthBeaconConsensus` | `validate_header_against_parent()` |

주요 책임: 블록 및 헤더 검증, 합의 규칙 적용, 포크 규칙 관리

9. 프리미티브 타입 (`crates/primitives/`)

| 디렉터리 | 역할 | 핵심 구조체 | 주요 함수 |

|---|---|---|---|

| `primitives/` | 기본 데이터 타입 | `Block`, `Transaction`, `Receipt` | `hash()`, `encode()` |

| `primitives-traits/` | 프리미티브 트레이트 | `NodePrimitives`, `Block` | `header()`, `body()` |

| `ethereum/primitives/` | Ethereum 특화 타입 | `EthPrimitives` | `block_hash()` |

주요 책임: 핵심 데이터 구조 정의, 직렬화/역직렬화, 해시 계산

10. 트라이 시스템 (`crates/trie/`)

| 디렉터리 | 역할 | 핵심 구조체 | 주요 함수 |

|----------|------|----------|----------|

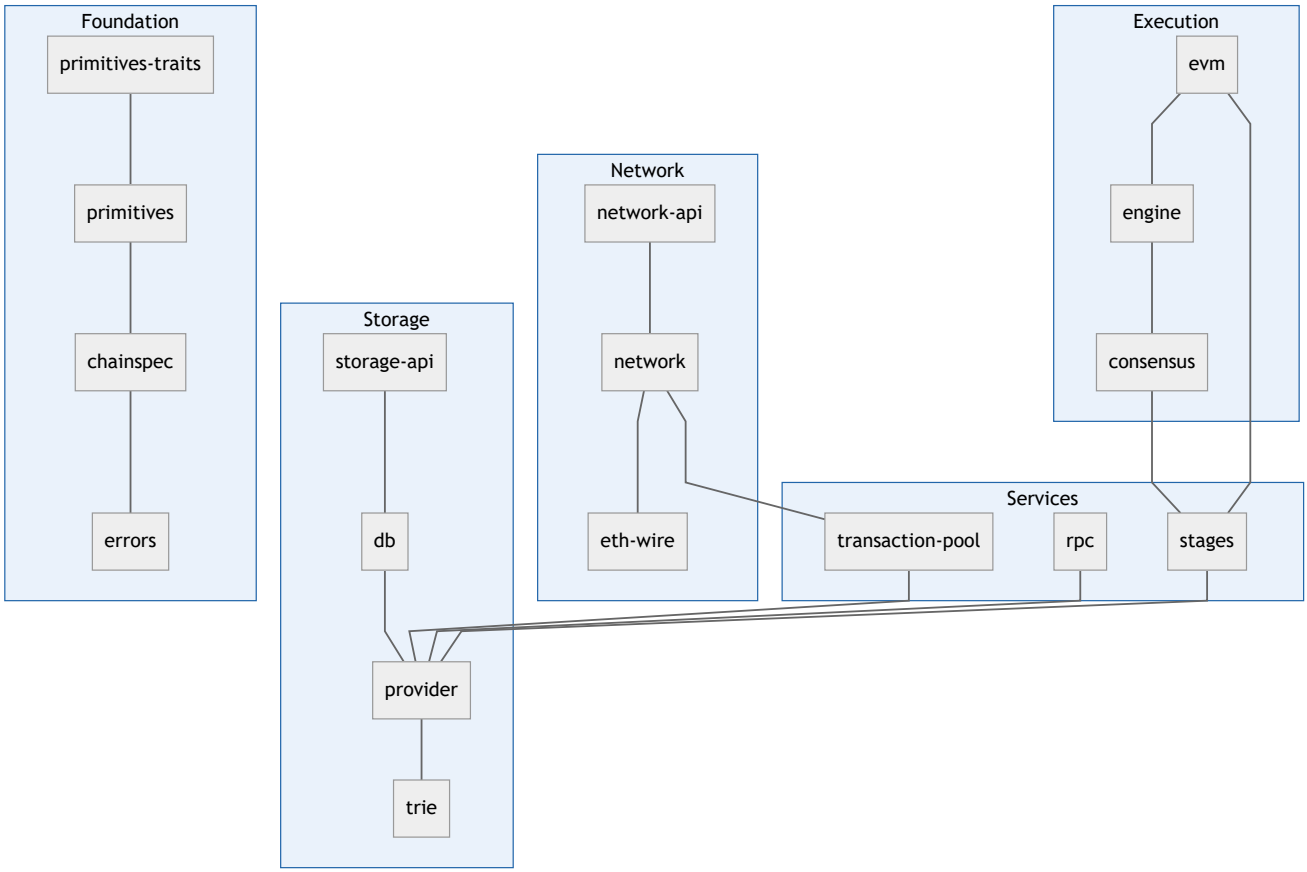
| `trie/trie/` | Merkle Patricia Trie | `Trie`, `TrieNode` | `root()`, `get()`, `insert()` |

| `trie/db/` | 트라이 데이터베이스 | `TrieDb`, `TrieCursor` | `trie_cursor()`, `account_trie()` |

| `trie/sparse/` | 희소 트라이 | `SparseTrie`, `SparseTrieBuilder` | `update_leaf()`, `root()` |

주요 책임: 상태 루트 계산, 머클 증명 생성, 트라이 업데이트

컴포넌트 간 의존성 다이어그램



핵심 데이터 흐름

1. 블록 동기화 흐름

1. **Network** → 피어로부터 블록 헤더/바디 수신
2. **Stages** → 단계별 검증 및 실행
3. **EVM** → 트랜잭션 실행 및 상태 업데이트

4. **Provider** → 데이터베이스에 영구 저장
5. **Trie** → 상태 루트 계산 및 검증

2. 트랜잭션 처리 흐름

1. **RPC/Network** → 트랜잭션 수신
2. **Transaction Pool** → 검증 및 메모리 풀 저장
3. **Payload Builder** → 블록 구성을 위한 트랜잭션 선택
4. **EVM** → 트랜잭션 실행
5. **Engine** → 블록 완성 및 합의 레이어 전달

3. RPC 요청 처리 흐름

1. **RPC Server** → 클라이언트 요청 수신
2. **RPC Handler** → 요청 파싱 및 라우팅
3. **Provider** → 데이터베이스 쿼리
4. **Response** → 결과 직렬화 및 응답

성능 최적화 특징

1. 메모리 관리

- **Arc/Rc 활용**: 데이터 공유를 통한 메모리 절약
- **지연 로딩**: 필요시에만 데이터 로드
- **압축 저장**: NippyJar를 통한 정적 파일 압축

2. 병렬 처리

- **비동기 I/O**: Tokio 기반 비동기 처리

- **병렬 검증**: 트랜잭션 및 블록 병렬 검증
- **파이프라인**: 동기화 단계별 병렬 실행

3. 캐싱 전략

- **LRU 캐시**: 자주 사용되는 데이터 캐싱
- **블룸 필터**: 빠른 존재 여부 확인
- **메모리 풀**: 트랜잭션 빠른 접근

확장성 고려사항

1. 모듈화 설계

- **트레이트 기반**: 구현체 교체 가능
- **플러그인 아키텍처**: 새로운 기능 추가 용이
- **설정 기반**: 런타임 동작 변경 가능

2. 네트워크 확장

- **다중 프로토콜**: discv4, discv5, DNS 지원
- **동적 피어 관리**: 네트워크 상황에 따른 적응
- **대역폭 제어**: 네트워크 리소스 효율적 사용

3. 스토리지 확장

- **다중 백엔드**: MDBX 외 다른 DB 지원 가능
- **샤딩 준비**: 미래 샤딩 지원을 위한 구조
- **아카이빙**: 오래된 데이터 효율적 저장

보안 고려사항

1. 메모리 안전성

- **Rust 언어**: 메모리 안전성 보장
- **타입 안전성**: 컴파일 타임 오류 검출
- **경계 검사**: 버퍼 오버플로우 방지

2. 네트워크 보안

- **피어 검증**: 악의적 피어 차단
- **메시지 검증**: 프로토콜 메시지 무결성 확인
- **DDoS 방어**: 요청 제한 및 우선순위 처리

3. 합의 보안

- **블록 검증**: 엄격한 합의 규칙 적용
- **상태 검증**: 상태 루트 무결성 확인
- **포크 처리**: 체인 재구성 안전 처리

이 아키텍처는 Reth가 고성능, 확장 가능하며 안전한 Ethereum 클라이언트로 동작할 수 있게 하는 기반을 제공합니다.